🔹 What is Abstraction?

💡 Definition:

Abstraction means **hiding complex internal details** and showing only the **necessary features** to the user.

It helps us **focus on what an object does**, not **how it does it**.

🧰 Real-Life Example:

When you drive a **car**, you use the steering wheel, accelerator, and brake —
you don't need to know how the engine works inside.

That's abstraction — the internal logic is hidden from you.

🧮 In Python:

- Abstraction can be done using **abstract classes** and **abstract methods** (from `abc` module).
- But even without using `abc`, **the concept** can be understood easily —
we show only what's needed and hide the rest inside the class.

✅ Example of Abstraction (Conceptual)

```
class Car:
    def start(self):
        print("Starting the car...")

    def drive(self):
        print("Car is moving...")

# We don't see the internal engine working, only the outer actions.
my_car = Car()
my_car.start()
my_car.drive()
```

Output:

```
Starting the car...
Car is moving...
```

Here, the user can start or drive the car,
but the complex working inside the engine is **hidden** — that's abstraction.

🔸 Advantages of Abstraction

1. Hides complex code details from the user.
2. Makes code easier to understand and maintain.
3. Improves security by exposing only the required parts of the object.

🔹 What is Polymorphism?

💡 Definition:

The word **Polymorphism** means *"many forms"*.
In programming, it means **the same function or method name can perform different actions** depending on the object that calls it.

🧰 Real-Life Example:

A **person** can have different roles:

- As a **student** in class
- As a **customer** in a shop
- As a **child** at home

👉 The same person (object) behaves differently depending on the situation.
That's **Polymorphism** — same name, different forms.

🧮 Example of Polymorphism

```
class Dog:
    def sound(self):
        print("Barks")

class Cat:
    def sound(self):
        print("Meows")

# Using the same function name for different objects
for animal in (Dog(), Cat()):
    animal.sound()
```

Output:

```
Barks
Meows
```

🧩 Here:

- Both `Dog` and `Cat` have the same method name `sound()`.
- But their **behavior is different** based on the object that calls it.
  This is **Polymorphism in action**.

🧠 Another Example

```
class Bird:
    def fly(self):
        print("Most birds can fly")

class Penguin(Bird):
```

```
    def fly(self):
        print("Penguins cannot fly")

# Same method name, different behavior
b1 = Bird()
b2 = Penguin()

b1.fly()
b2.fly()
```

Output:

```
Most birds can fly
Penguins cannot fly
```

🔸 Advantages of Polymorphism

1. Makes code more flexible and extensible.
2. Allows using the same function name for different object types.
3. Supports code reusability.

🧾 Summary Table

Concept	Meaning	Real-Life Example	Python Example	Benefit
Abstraction	Hiding complex details	Car driving, TV remote	Hiding inner logic using methods	Simplifies complex code
Polymorphism	Same name, different behavior	Person with many roles	`sound()` for Dog & Cat	Makes code flexible

---

🧩 Practice Tasks

1. **Create a class `Shape`** with a method `area()` and subclasses like `Circle` and `Rectangle` that calculate area differently.
   → Demonstrates **Polymorphism**.

2. **Create a class `Phone`** with methods like `call()` and `message()` showing only necessary actions, hiding the internal logic.
   → Demonstrates **Abstraction** conceptually.